

## WI-TSPIPE-002 — Running an analysis on clinical-23

|                    |                                                                                        |
|--------------------|----------------------------------------------------------------------------------------|
| Document number    | WI-TSPIPE-002                                                                          |
| Version            | 2.0                                                                                    |
| Applies to         | nf-core-tspipe v1.0.1, clinical-23 installation, Twist myeloid panel                   |
| Parent document    | SOP-TSPIPE-001                                                                         |
| Purpose            | Run a batch of samples end to end on clinical-23, from FASTQ files to reviewed reports |
| Attention required | ~15 minutes to prepare and launch, 2-3 hours unattended, ~15 minutes to review         |

Use this for every run. Parts A to C apply to any batch; Part D applies additionally when the run is a validation or re-qualification run against the reference cohort.

Follow the steps in order. Each has an expected result. **If a step does not give the expected result, stop there and record what you saw** — do not adjust configuration, paths or scripts to get past it. Everything runs as `patkarlab-clinical` on `ln1`.

### Part A — Prepare

#### A1. Log in to the login node

```
ssh patkarlab-clinical@10.100.95.23
hostname -s
```

**Expected:** `ln1`.

Runs are launched from here and never submitted to the queue. Three steps of the pipeline need internet access and compute nodes have none; a run launched as a PBS job produces reports with silently empty annotation sections.

#### A2. Go to the pipeline and confirm the version

```
cd ~/pipelines/nf-core-tspipe-v1.0.0
git describe --tags
git status --short
```

**Expected:** `v1.0.1`, and `git status` listing only untracked files (your samplesheets, `params_clinical23.yaml`, stray `.out` files).

**Stop if:** any line begins with `M` — a tracked file of the frozen release has been modified. Record which file.

Note: the directory is named `...-v1.0.0` but holds `v1.0.1`. The tag is what identifies the version, not the directory name.

#### A3. Verify the installation

```
bash tools/verify_install.sh -profile clinical23,singularity \
-c conf/twist_apply.config -params-file params_clinical23.yaml 2>&1 | tail -8
```

**Expected:** ends with `VERIFY PASSED`. Warnings about three unused images (`local-mocha`, `local-tspipe-host-v1`, `zhkddocker-flt3_itd_ext`) and about `/goast/hemat_data/nfcore_runs` not being writable are normal.

**Stop if:** any line begins with `[FAIL]`, or the output ends `VERIFY FAILED`.

This checks the toolchain, all fourteen container images against their checksums, and every reference file against the manifest. Run it before every batch; it takes under a minute and it is the only thing standing between a corrupted reference and a wrong clinical result.

#### A4. Stage the FASTQ files

Put the batch's FASTQ files in one directory under `/scratch`, not in your home directory:

```
BATCH=/scratch/patkarlab-clinical/batch_$(date +%Y%m%d)
mkdir -p $BATCH
# copy or move the FASTQ files into $BATCH, then:
ls -la $BATCH/*.fastq.gz | wc -l
du -sh $BATCH
```

**Expected:** an even number of files (one R1 and one R2 per sample) and a size consistent with the batch — roughly 3 GB per sample for this panel.

Files must be gzipped FASTQ (`.fastq.gz`). Do not decompress them.

#### A5. Build the samplesheet

The samplesheet tells the pipeline which samples to run, where their reads are, and each sample's sex. It is a CSV file with a header line and one line per sample.

##### Format

```
sample,fastq_1,fastq_2,sex
26ABC123-Twist,/scratch/patkarlab-clinical/batch_20260915/26ABC123_R1.fastq.gz,/scratch/patkarlab-clinical/batch_20260915/26ABC123_R2.fastq.gz,male
26ABC124-Twist,/scratch/patkarlab-clinical/batch_20260915/26ABC124_R1.fastq.gz,/scratch/patkarlab-clinical/batch_20260915/26ABC124_R2.fastq.gz,female
26ABC125-Twist,/scratch/patkarlab-clinical/batch_20260915/26ABC125_R1.fastq.gz,/scratch/patkarlab-clinical/batch_20260915/26ABC125_R2.fastq.gz,unknown
```

| Column  | Content                 | Rules                                                                                                                                                                                                                             |
|---------|-------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| sample  | The sample identifier   | Becomes the output directory name and appears in every report and table. Use the laboratory accession exactly. Letters, digits, hyphens and underscores only — no spaces, no commas, no slashes. Must be unique within the batch. |
| fastq_1 | Read 1 file             | Absolute path. Must exist and be readable. <code>.fastq.gz</code> .                                                                                                                                                               |
| fastq_2 | Read 2 file             | Absolute path, the mate of <code>fastq_1</code> .                                                                                                                                                                                 |
| sex     | male, female or unknown | Selects the sex-matched panel of normals for copy-number calling. Supply it when known; unknown is permitted and the pipeline reports inferred sex from the sex-check step.                                                       |

The header line must be exactly `sample,fastq_1,fastq_2,sex`.

## Generate it

For files named `<sample>_R1.fastq.gz` / `<sample>_R2.fastq.gz`:

```
cd ~/pipelines/nf-core-tspipe-v1.0.0
BATCH=/scratch/patkarlab-clinical/batch_20260915      # your directory from A4
SHEET=batch_20260915.csv                             # name it after the batch

{ echo "sample,fastq_1,fastq_2,sex"
  for r1 in $(BATCH/*_R1*.fastq.gz); do
    r2=${r1/_R1/_R2}
    s=$(basename "$r1"); s=${s%%_R1*}
    if [ ! -f "$r2" ]; then echo "NO MATE for $r1" >&2; continue; fi
    echo "$s,$r1,$r2,unknown"
  done
} > $SHEET

cat $SHEET
```

If your files use a different naming convention (for example `_1.fastq.gz` / `_2.fastq.gz`, or Illumina's `S1_L001_R1_001.fastq.gz`), adjust the two `_R1` / `_R2` patterns accordingly and check the resulting sample names before continuing.

**Then set the sex column.** The generator writes `unknown` for every sample; edit the file and replace it where the sex is known:

```
nano $SHEET      # or vi
```

## Check it before running

```
SHEET=batch_20260915.csv

head -1 $SHEET                                     # header exactly right?
grep -c '$\r' $SHEET                               # Windows line endings?
awk -F, 'NR>1{print $1}' $SHEET | sort | uniq -d     # duplicate sample names?
awk -F, 'NR>1{print $4}' $SHEET | sort | uniq -c     # sex values sane?
awk -F, 'NR>1{print $2"\n"$3}' $SHEET | while read f; do [ -f "$f" ] || echo "MISSING $f"; done
echo "samples: $(( $(grep -c . $SHEET) - 1 ))"
```

### Expected:

- the header line reads `sample,fastq_1,fastq_2,sex`;
- the carriage-return count is 0;
- the duplicate check prints nothing;
- the sex values are only `male`, `female` or `unknown`;
- no `MISSING` lines;
- the sample count is what you expect.

### Common problems:

| Symptom                        | Cause                                                          | Fix                                                                                                                |
|--------------------------------|----------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------|
| Carriage-return count is not 0 | The file was edited on Windows or exported from Excel          | <code>sed -i 's/\r\$//' \$SHEET</code>                                                                             |
| A sample name contains a space | Taken from a filename with a space                             | Rename the FASTQ files and regenerate                                                                              |
| MISSING lines                  | Wrong directory, a typo, or a file that did not finish copying | Correct the path; confirm the transfer completed                                                                   |
| Duplicate sample names         | Two lanes of the same sample                                   | Merge the FASTQ files first, or give the lanes distinct names — the pipeline treats each line as a separate sample |
| A sample has no mate           | R2 absent or differently named                                 | Locate the mate before running                                                                                     |

## A6. Pre-run checks

```
df -h /scratch | tail -1
pgrep -u patkarlab-clinical -f nextflow | wc -l
qstat -u patkarlab-clinical | grep -c nf-TSPIPE
```

**Expected:** free space of at least 28 GB per sample; 0 Nextflow processes; 0 tspipe jobs.

Other users share this cluster, so `qstat` may show unrelated jobs; only `nf-TSPIPE` jobs are ours.

**Stop if:** a Nextflow process is already running. It holds a session lock and a second run against the same work directory will fail.

## Part B — Run

### B1. Launch

```
cd ~/pipelines/nf-core-tspipe-v1.0.0
RUN=batch_20260915 # a short name for this run; used for the output paths
SHEET=batch_20260915.csv
bash tools/launch_clinical23.sh $SHEET $RUN
```

**Expected:** the launcher performs its checks and prints a block like:

```
[...] VariantValidator preflight: OK
[...] pipeline /home/patkarlab-clinical/pipelines/nf-core-tspipe-v1.0.0 (v1.0.1)
[...] samples 8 from batch_20260915.csv
[...] profile clinical23,singularity
[...] outdir /scratch/patkarlab-clinical/batch_20260915
[...] launched. Monitor with: ...
```

**Stop if:** it refuses. The refusal states the reason — wrong machine, inside a PBS job, missing samplesheet or parameter file, a FASTQ that does not exist, insufficient space, a run already active, or an unreachable VariantValidator endpoint. Each is a real condition, not an obstacle to work around.

You can log out now; the run continues.

Do not launch with `launch_tspipe.sh`. That is the development host's launcher: it selects that host's profile, so every reference resolves under `/goast`, which does not exist here, and the run fails immediately with missing files.

### B2. Monitor

Check every 20–30 minutes:

```
RUN=batch_20260915 # re-set this if you logged out
grep -cE "Submitted process" /tmp/${RUN}.log
grep -E "ERROR|terminated with" /tmp/${RUN}.log | tail -3
qstat -u patkarlab-clinical | grep -c nf-TSPIPE
```

**Expected progression:** the task count climbs — roughly 56 tasks per sample — with no `ERROR` lines, and jobs coming and going in the queue.

Two stages look stalled and are not:

- **VARSCAN** runs `samtools mpileup` for 20–30 minutes per sample and writes a temporary file of one to two gigabytes. The task count moves little during this.
- **VARIANT\_VALIDATOR** is deliberately serialised — one request at a time against a public service — so it progresses slowly with no parallelism.

Jobs in state `Q` mean the cluster is busy with other people's work. Normal; the run is not stuck.

**Stop if:** a line reads `Error executing process`. Record the process name and the work directory path it prints.

### B3. Confirm completion

```
RUN=batch_20260915
grep -E "Cleanup complete" /tmp/${RUN}.log
grep -ciE "error executing" /tmp/${RUN}.log
pgrep -u patkarlab-clinical -f nextflow | wc -l
ls /scratch/patkarlab-clinical/${RUN}/
```

**Expected:** the `Cleanup complete`. Final per-sample layout: ... line; an error count of 0; no Nextflow process; and a listing showing one directory per sample plus `assets`, `cohort_index.html` and `${RUN}_reports.zip`.

A run that ends without the completion line did not finish. Treat it as a failure and see "If something goes wrong".

## Part C — After the run

### C1. Outputs

Under `/scratch/patkarlab-clinical/<RUN>/`:

| Path | Content |
|------|---------|
|------|---------|

|                                                          |                                                     |
|----------------------------------------------------------|-----------------------------------------------------|
| <sample>/clinical/<sample>.somaticseq.clinical.final.tsv | Final annotated clinical variant table              |
| <sample>/clinical/<sample>.somaticseq.filtered.tsv       | Full filtered variant table                         |
| <sample>/clinical/<sample>_report.html, _dashboard.html  | Reports for review                                  |
| <sample>/clinical/<sample>_igv_report.html               | Read-level visualisation of reported variants       |
| <sample>/clinical/<sample>.final.bam                     | Analysis-ready alignment                            |
| <sample>/clinical/cnv/                                   | CNV consensus, BAF, PURPLE, CNVkit, DECoN, reconCNV |
| <sample>/<sample>_report.zip                             | Self-contained per-sample bundle for distribution   |
| qc/, sex_check/                                          | Quality control outputs                             |
| cohort_index.html, <RUN>_reports.zip                     | Cohort index and combined bundle                    |

## C2. Review before release

```
RUN=batch_20260915
ls /scratch/patkarlab-clinical/${RUN}/*/clinical/*_report.html | wc -l
ls /scratch/patkarlab-clinical/${RUN}/*/clinical/*oncokb_cache.json 2>/dev/null | wc -l
for d in /scratch/patkarlab-clinical/${RUN}/*; do s=$(basename $d); printf "%-24s %s variants\n" "$s" "$(( $(wc -l < $d/clinical/${s}.somaticseq.clinical.final.tsv 2>/dev/null) - 1 ))"; done
```

Confirm: one report per sample; one OncoKB cache per sample; a plausible variant count for each; QC metrics within laboratory acceptance criteria; and, opening `cohort_index.html` and a sample dashboard in a browser, that the variant tables, the CNV tab and the annotation columns are populated.

**Empty GeneBe, MobiDetails or OncoKB sections indicate a network or credentials fault, not an absence of findings.** Do not release such a report; record it and escalate.

## C3. Records and cleanup

Retain the run log and the output directory per laboratory policy:

```
RUN=batch_20260915
cp /tmp/${RUN}.log /scratch/patkarlab-clinical/${RUN}/
du -sh /scratch/patkarlab-clinical/${RUN} /scratch/patkarlab-clinical/work_${RUN}
```

Once outputs are verified and archived, the work directory may be deleted. This frees most of the space and prevents any later resumption of the run:

```
rm -rf /scratch/patkarlab-clinical/work_${RUN}
```

Outputs are independent files, not links, and are unaffected.

## Part D — Validation and re-qualification runs only

Perform this part when the run is the eight-sample reference cohort — after any change to the installation, and periodically to confirm the system still behaves as qualified.

The inputs already exist on this machine: FASTQ files in `/scratch/patkarlab-clinical/twist_val_fastq/` and the samplesheet `twist_val_8_clinical23.csv` in the pipeline directory. Parts A4 and A5 are therefore not needed; use that samplesheet as it is.

```
cd ~/pipelines/nf-core-tspipe-v1.0.0
RUN=val_$(date +%Y%m%d)
bash tools/launch_clinical23.sh twist_val_8_clinical23.csv $RUN
```

When it completes, compare against the accepted reference result:

```
python3 tools/compare_runs.py \
  --a /scratch/patkarlab-clinical/tspipe_run8_c23 \
  --b /scratch/patkarlab-clinical/${RUN} \
  --label-a accepted --label-b new \
  --out /scratch/patkarlab-clinical/${RUN}_vs_accepted.md
sed -n 1,12p /scratch/patkarlab-clinical/${RUN}_vs_accepted.md
grep '^### ' /scratch/patkarlab-clinical/${RUN}_vs_accepted.md | sed -E 's/26CGH[0-9]+-TwistMyVal/<S>/g' | sort | uniq -c | sort -rn | head
-10
```

**Expected:** roughly 1,200 identical files, with differences confined to:

- timestamped logs — `amber.log`, `cobalt.log`, `purple.log`, `_hsmetrics.txt`, `quickcheck.txt`;
- files that embed a creation time — `.zip`, `.pdf`, `.rds`, and the HTML reports;
- annotation caches (`_genebe_cache.json`, `_mobidetails_cache.json`);
- `_oncokb_cache.json` present in the new run and absent from the accepted one. This is expected: the OncoKB token was installed on 2026-09-11, after the reference run was produced.

**Must be identical.** If any of these appear under "Real text differences", stop and record it:

- `<sample>/clinical/<sample>.somaticseq.clinical.final.tsv`
- `<sample>/clinical/<sample>.somaticseq.filtered.tsv`
- `<sample>/clinical/cnv/consensus/*.genes.tsv`

- `<sample>/clinical/cnv/baf/*.summary.tsv`
- the PURPLE, CNVkit, DECoN and GATK CNV tables

A difference in any clinical table means the installation has changed since qualification. Retain the comparison report as the qualification record.

## If something goes wrong

Nextflow names the failing process and its work directory. That directory holds `.command.sh` (the exact command), `.command.log` (its output), `.command.run` (the wrapper, including the scheduler directives and the container invocation) and `.exitcode`.

```
RUN=batch_20260915
grep -A25 "Error executing process" /tmp/${RUN}.log | head -35
```

| What you see                               | What it means                                                     | What to do                                                                                                                                                                                                 |
|--------------------------------------------|-------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| exit status (137)                          | Task killed, usually for exceeding its memory allocation          | Record the process name; escalate                                                                                                                                                                          |
| Process requirement exceeds available CPUs | A step running on In1 asked for more CPUs than the local pool has | Escalate                                                                                                                                                                                                   |
| Unable to acquire lock on session          | A previous Nextflow is still exiting                              | Wait until no Nextflow process remains, then resume                                                                                                                                                        |
| No such file or directory: /goast/...      | The run used the development host's profile                       | The wrong launcher was used; re-launch per B1                                                                                                                                                              |
| VariantValidator task failed               | External service unreachable or rate-limiting                     | Three automatic retries are configured. If it still fails, test with <code>curl -s -m 10 -o /dev/null -w '%{http_code}\n' https://rest.variantvalidator.org/</code> ; if the service is down, resume later |
| Reports with empty annotation sections     | The dashboard ran without internet access                         | Escalate — results are incomplete and must not be released                                                                                                                                                 |
| A task fails once, succeeds on resume      | Transient cluster fault                                           | Acceptable. Record it                                                                                                                                                                                      |
| Jobs sit in state Q                        | The cluster is busy with other users' work                        | Normal. Wait                                                                                                                                                                                               |

**To resume an interrupted run** — same run name, same samplesheet, same work directory:

```
RUN=batch_20260915
bash tools/launch_clinical23.sh batch_20260915.csv $RUN -resume
```

Completed tasks are reused and reported as `Cached process`; only what failed re-runs.

Do not modify the pipeline, the container images, the reference data or the parameters to work around a failure. A result obtained by an undocumented change cannot be used clinically and invalidates the installation's qualification.

## Notes

**This machine runs two pipelines.** `~/pipelines/nf-core-tspipe-v1.0.0` is the frozen clinical release (v1.0.1), used for the Twist myeloid panel and covered by this instruction. `~/pipelines/nf-core-tspipe` is a separate, older tree that serves the MYOPOOL panel. They are independent; nothing in this document applies to the MYOPOOL tree, and nothing here should be run from it.

**Why the launcher.** It supplies the site profile, the panel overlay, the site parameter file and the annotation endpoint, and refuses to start from a batch job or from the wrong machine. A run started by other means may silently use another site's configuration or lose its annotation.

**Escalation.** Provide: the run name, `/tmp/<run>.log`, the failing process name and its work directory, and the output of `bash tools/verify_install.sh ...` from step A3.